

— SYSTEM OVERVIEW · NASDAQ TOTALVIEW-ITCH 5.0

How Animus Core ingests market data.

A plain look at the pipeline, the numbers measured behind it, and where its scope ends — prepared for infrastructure and engineering review.



7.80M msgs/sec sustained (Linux)

98-130 ns p99, parse + enqueue

0 heap allocations recorded

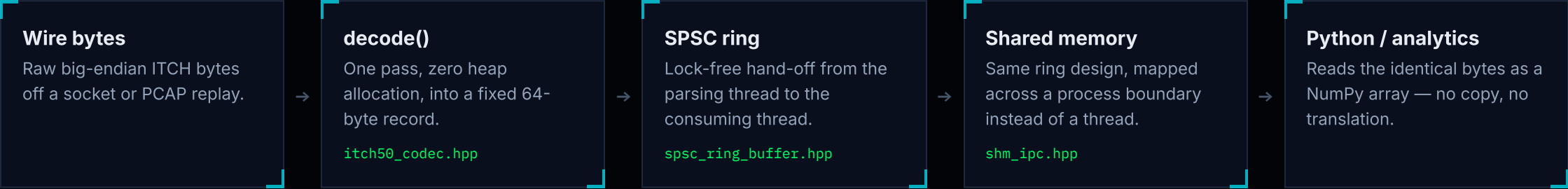
2 independent platforms verified

A feed handler's only job is to not become the bottleneck.

Any component that sits between an exchange feed and your book, risk engine, or analytics layer has one basic obligation: keep up with message bursts without adding its own delay, and without needing constant engineering attention once it's running. These are the three considerations that actually matter when evaluating a component like this — not a sales pitch, just the real questions an infrastructure team asks.

CONSIDERATION 1 Predictable latency under load Every message is parsed with no memory allocation, so one slow allocation can't stall every message queued behind it. See slide 04.	CONSIDERATION 2 Resource footprint The producer and consumer threads benefit from dedicated cores, but neither the decoder nor the ring buffer has any other runtime dependency. See slide 05.	CONSIDERATION 3 Integration surface Decoded records are exposed over shared memory and a plain C interface — existing Python tooling attaches to it; it doesn't get replaced by it. See slide 06.
--	--	---

Five stages, from wire to your analytics process.



MESSAGE MIX EXERCISED IN THE VERIFIED RUN — 10,000,000 MESSAGES



This mix is a seeded synthetic generator weighted to resemble a liquid single-name symbol's real order flow — heavy on adds, deletes and executions, light on system events. **It is not a captured live feed**; every run uses the identical seed so results are directly comparable across platforms.

No message ever waits on a memory allocation.

Every ITCH message — a 12-byte system event or a 40-byte attributed order — is read directly off the wire bytes into one uniform 64-byte record. Nothing in that step touches the heap.

A general-purpose allocator is not instant in the worst case: one unlucky allocation doesn't just slow down the message that triggered it, it stalls every message waiting behind it on a single-file queue. Removing the allocator from this path removes that failure mode, rather than just making it rare. This is instrumented, not assumed — the benchmark overrides the global allocator and counts every call.

- S System Event
- A Add Order
- F Add Order + MPID
- E Order Executed
- C Executed + Price
- X Order Cancel
- D Order Delete
- U Order Replace

WIRE LAYOUT — 'A' ADD ORDER, 36 BYTES

OFFSET	FIELD	BYTES	MEANING
0	message_type	1	ASCII 'A'
1	stock_locate	2	Symbol routing key
3	tracking_number	2	Session sequence
5	timestamp	6	ns since midnight
11	order_reference_number	8	Order identity
19	buy_sell_indicator	1	'B' or 'S'
20	shares	4	Order size
24	stock	8	Ticker, space-padded
32	price	4	4 implied decimals

Field offsets follow the public NASDAQ ITCH 5.0 spec —
adapters/itch50/include/itch50_messages.hpp

One writer, one reader. Ordinary memory rules do the rest.

The parsing thread and the consuming thread pass records through a shared ring, but neither ever waits on a lock. Each side publishes how far it has gotten; the CPU's own rules for when a write becomes visible to another core handle the rest, at the cost of an ordinary memory write.

The two counters are deliberately kept on separate 64-byte cache lines. Without that separation, the producer updating its counter would invalidate the same line the consumer is reading from, forcing a cross-core cache round-trip on every message.

Scope, stated plainly: this ring is one producer, one consumer, by design — the fastest topology for a direct hand-off. Fanning out to several independent consumers uses a separate broadcast-ring variant already in the codebase, not this one.

CACHE-LINE LAYOUT — 64 BYTES EACH

PRODUCER-OWNED LINE



— separate cache lines: nothing shared, nothing to invalidate —

CONSUMER-OWNED LINE



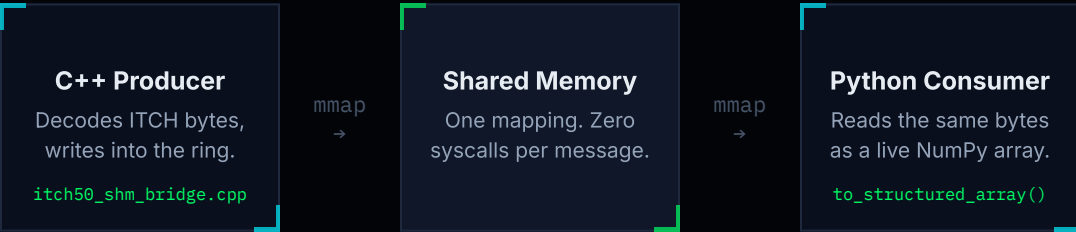
RING PAYLOAD — 4 OF 1,048,576 SLOTS



Padding cost: 56 idle bytes per line. Coherency traffic avoided: on every message. — animus-eval-kit/include/spsc_ring_buffer.hpp

Shared memory, not a serialization layer.

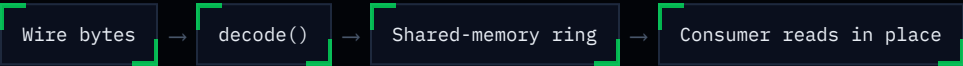
Getting decoded records into a Python risk or analytics process normally means serializing them, sending them over a socket, and deserializing them on the other end. Instead, both processes map the same physical memory page and read it directly — like two programs looking at the same whiteboard instead of mailing each other letters.



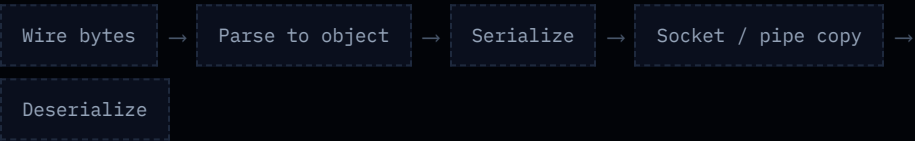
```
# zero-copy: a view over shared memory, not a copy
channel = SharedSchemaChannel.open("animus_itch50_demo")
arr = to_structured_array(channel)
assert arr.flags["OWNDATA"] is False
```

STRUCTURAL COMPARISON – HOP COUNT ONLY

THIS PIPELINE



A TYPICAL SERIALIZED IPC PATH



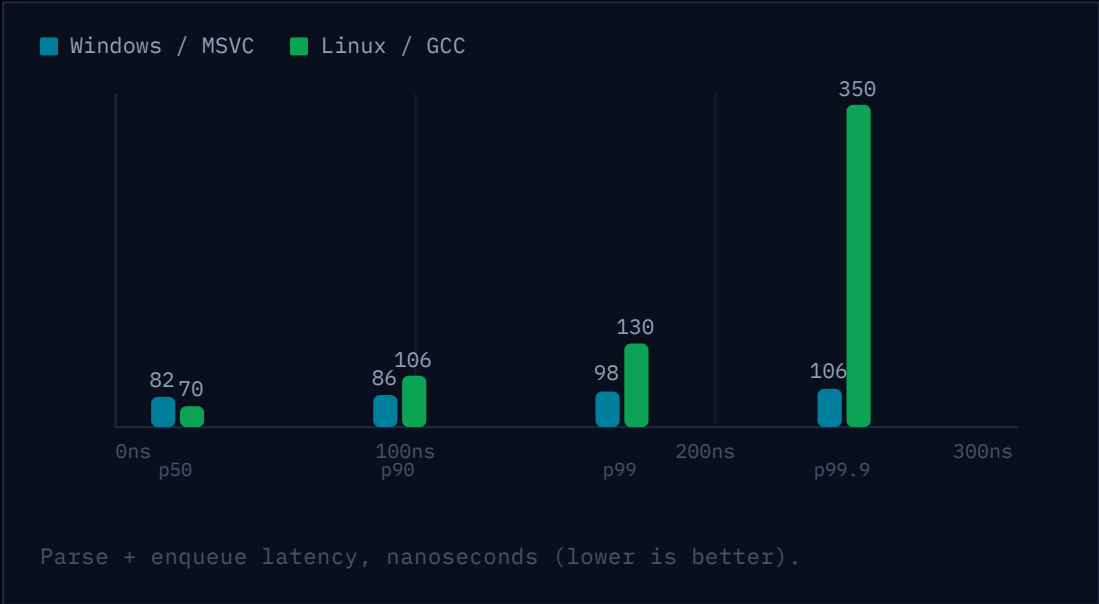
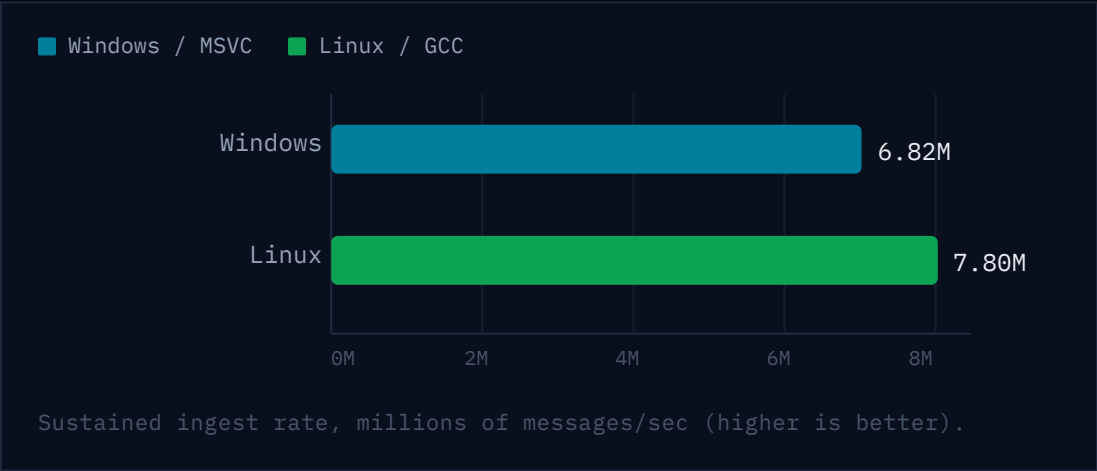
This compares hop count only, not measured latency – the traditional path shown here is illustrative and **was not independently benchmarked** in this study. See slide 08.

Because the exported interface is plain C with only fixed-width types crossing it, the same surface is reachable from any language with a C FFI. Python via ctypes is the path this repository implements and benchmarks today.

Two platforms. One 10,000,000-message run each.

Every figure below is a real bench_itch_ingest run, not a projection — reproducible on your own hardware with the command in slide 08.

6.82M msgs/sec, sustained WINDOWS · MSVC 19.51	7.80M msgs/sec, sustained LINUX · GCC 15.2	0 / 10M heap allocations BOTH PLATFORMS	0 / 10M sequence corruptions BOTH PLATFORMS
---	---	--	--



Neither run used a dedicated, core-isolated rig (no isolcpus/nohz_full). p50/p90/sustained throughput are the stable, representative figures; p99.9/max tail latency is what varies most on an untuned host — see slide 08.

✓ VERIFIED: 0 decode failures · 0 corrupted frames · 0 hot-path heap allocations — both platforms, every run

What this system is today — stated plainly.

Every constraint below is a scope choice or an open item, not a hidden defect. Each traces to a real file your own engineers can open.

DECODER GRADE	Reference-grade, not vendor-certified. Verify field offsets against your own copy of the official ITCH 5.0 specification before pointing this at a production feed.
BOOK STATE	No order-book reconstruction. This normalizes each message into a flat, uniform record for ring transport — it does not build or maintain a live book.
TEST RIG	Not a core-isolated benchmark host. Both verified runs used a shared development machine. Sustained throughput and p50/p90 are stable; p99.9/max tail latency should be re-measured on your target hardware.
RING TOPOLOGY	One producer, one consumer per ring. Multi-consumer fan-out uses a separate broadcast-ring design already present in the codebase, not the SPSC ring described here.
MPID FIELD	Parsed, not carried forward. The Attribution (MPID) field on 'F' messages is read off the wire but not currently included in the normalized record, by scope choice.

Nothing on this page is asserted without a file to check it against — see docs/ARCHITECTURE.md and adapters/itch50/README.md.

— CONCLUSION

What the system does, in one paragraph.

Animus Core's ITCH 5.0 pipeline takes raw exchange bytes and gets them into a normalized, analyzable form **without allocating memory**, **without locking**, and **without copying data** across a process boundary more than once. That behavior has been measured, not assumed, on two independent operating systems and toolchains — ten million messages per run, zero recorded corruption, zero recorded allocation. It is a reference-grade implementation: the architecture is proven, and the scope noted on the previous page is the honest boundary of what has been validated so far.

docs/ARCHITECTURE.md	Full technical deep-dive — every code excerpt quoted verbatim from source.
adapters/itch50/	The decoder, wire structs, benchmark harness, and shared-memory bridge.
scripts/verify_env.sh	Checks the host, then builds and runs the benchmark — reproduces slide 07.

